NTNU

Det skapende universitet

TDT4110 Informasjonsteknologi grunnkurs:

Tema:

- Tekststrenger
- Lese og skrive data med numpy:
  - loadtxt, savetxt

Dag Olav Kjellemo

Tekststrenger
Læringsmål og pensum

- Mål
  - Lære om
    - Grunnleggende operasjoner på strenger
    - Å skive/*slice* strenger
    - Teste, søke i og manipulere strenger
    - Lagre og lese data fra tekst-filer

Tekststrenger (string)

- I Python er en (*tekst-)streng* en egen datatype.
- Tre måter å angi tekst eksplisitt:

```
'tekst', "abc\n123", '''literal
string'''
```

- Ikke-muterbar!
- Det er en type *sekvens*, så vi kan mange av tingene som lister/tupler:
  - `'abcd'[2]   gir c`
  - `for t in 'abcd':  # gå gjennom hvert tegn`
    `  # gjør noe med t`

Grunnleggende strengoperasjoner

- Prosessering av tekst er en viktig del av programmering, fordi så mye representeres som tekst.
- Fordi tekst er viktig, så har Python mange metoder for å jobbe med strenger: undersøke egenskaper og manipulere/lage strenger

Tilgang til enkelttegn i en streng

- For å få tilgang til de enkelte tegn i en streng
  - Bruk en for-løkke
    - Format: `for character in string:`
    - Nyttig når man skal iterere over en hel streng, eksempelvis for å telle antall forekomster av et visst tegn i den
  - Bruk indeksering
    - Hvert tegn har en indeks som spesifiserer dens plassering i strengen, og begynner med 0
    - Format: `character = my_string[i]`

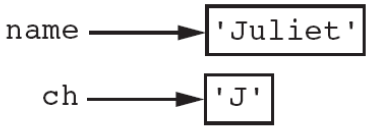
kode: sjekk_streng.py

NTNU
Det skapende universitet

Iterasjon over strengen “Juliet”

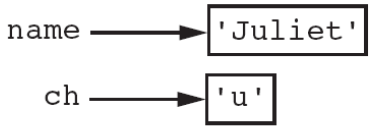
1st Iteration

```
for ch in name:
    print(ch)
```



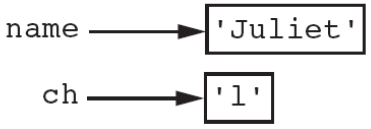
2nd Iteration

```
for ch in name:
    print(ch)
```



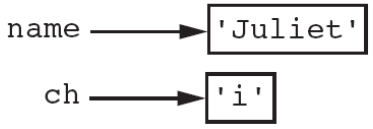
3rd Iteration

```
for ch in name:
    print(ch)
```



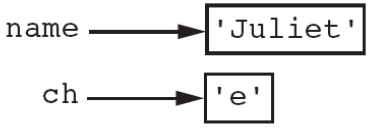
4th Iteration

```
for ch in name:
    print(ch)
```



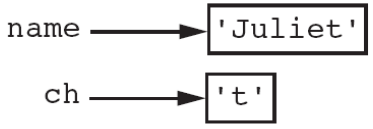
5th Iteration

```
for ch in name:
    print(ch)
```



6th Iteration

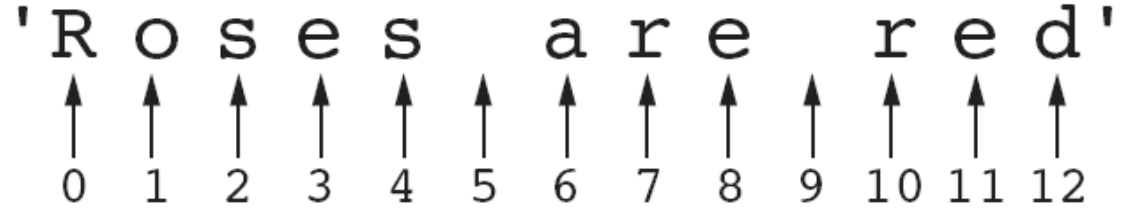
```
for ch in name:
    print(ch)
```



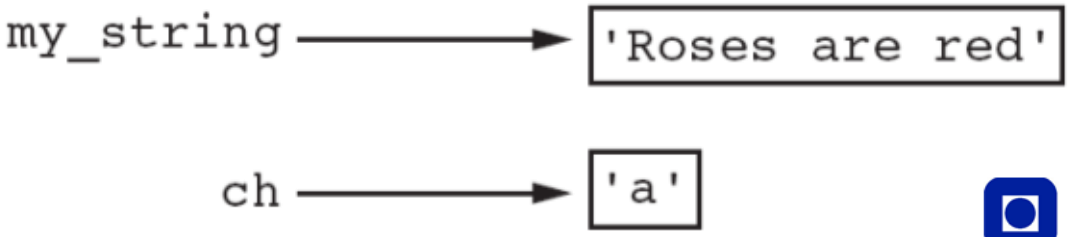
NTNU
Det skapende universitet

Tilgang til enkelttegn i en streng (forts.)

```
my_string = 'Roses are red'
```




```
ch = my_string[6]
```



NTNU
Det skapende universitet

Tilgang til enkelttegn i en streng (forts.)

- IndexError-exception utløses hvis:
  - Du prøver å bruke en indeks som er utenfor strengens rekkevidde
    - Dette skjer nok hvis du forsøker å iterere videre når strengen slutter
- Funksjonen len(string) kan brukes til å finne strengens lengde på samme måte som for lister
  - Snedig for å hindre at man itererer forbi strengens slutt

Oppgave:

- Lag funksjon `telle_tegn` som sjekker antall ganger et gitt tegn befinner seg i en tekst.

kode: sjekk_streng.py

NTNU
Det skapende universitet

Konkatenering (sammenkjeding) av strenger

Konkatenering av strenger

- kontatenering: å tilføye en streng ved slutten av en annen
  - Bruk operatoren + til å lage en streng som er en kombinasjon av dens operander
  - Den utvidede tilordningsoperatoren += kan også brukes til å kontatenere strenger
    - Operanden på venstre side av operatoren += må være en eksisterende variabel; i motsatt tilfelle utløses en exception

Tekststrenger og konkatenering (sammensetn)

Men man kan sette sammen flere tekststrenger ved å bruke + i mellom tekststrengene:

```
tekst = 'Dette '+ 'er '+'en '+'test'
# Gir 'Dette er en test'
```

Kan også sette sammen tekststrenger fra variabler:

```
nytekst = 'Her kommer: '+tekst
# Gir 'Her kommer: Dette er en test'
```

Hvis variabler med tall skal settes inn en streng, bruk str(variabel) som gjør om variabel til tekststreng:

```
tekst = 'Tallet er: '+str(tall)
```

Strenger er ikke-muterbare

- Når de først er laget, kan de ikke endres
  - kontatenering endrer ikke faktisk noen av strengene, men lager tvert i mot en ny streng, og tilordner denne til en av de allerede eksisterende variablene
  - Man kan **ikke** bruke et uttrykk som dette:

```
string[index] = new_character
tekst[3] = ”Klare” # Gir feilmelding!!!
```

    - Dette vil gi en feilmelding

Strenger er ikke-muterbare (forts.)

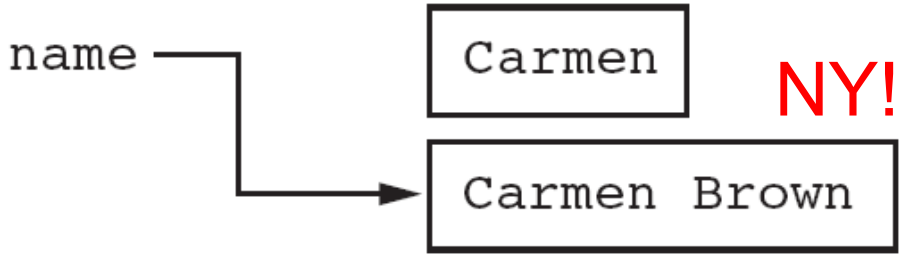
- Tekststrengen `'Carmen'` blir tilordnet `name`:

```
name = 'Carmen'
```



- Tekststrengen `'Carmen Brown'` blir tilordnet `name`.
  Merk! Et nytt tekst objekt blir laget!!!

```
name = name + ' Brown'
```



NTNU
Det skapende universitet

Oppgave: finn tegn i tekst



- Skriv Python-koden til funksjonen `skriv_tegn_indeks`, som tar inn en parameterne `tekst` og `tegn`, og returnerer alle indeksene der `tegn` befinner seg i strengen `tekst`.

kode: skriv_tegn_indeks.py

Slicing (skiving/utvalg) av strenger

Slicing av strenger (som for lister)

- Slice: spennvidde av enheter tatt fra en strengsekvens, kjent som en *substring*
  - Format: `string[start : end : steg]`
    - Uttrykket vil returnere en streng som har en kopi av tegnene fra start til, men ikke medregnet end
    - Hvis start er uspesifisert, antas indeks 0
    - Hvis end er uspesifisert, antas indeks len(string)

```
tekst = 'Dette er en test'
tekst[0:3]  # Gir 'Det'. Samme som tekst[:3]
tekst[12:16]  # Gir 'test'. Samme som tekst[12:]
tekst[::2]  # Gir 'Dtee nts'. (annenhvert tegn)
```

Testing, søking og manipulering av strenger

Kapittel 8.3

Testing, søking og manipulering av strenger

- Du kan bruke operatoren in til å avgjøre om en streng inneholdes av en annen streng (samme som lister)
  - Generelt format: `streng1 in streng2`
    - streng1 og streng2 kan være *string literals* eller variable som refererer til strenger
  - Tilsvarende kan du bruke operatoren not in til å avgjøre om en streng ikke er inneholdt av en annen streng

Strengmetoder

- Strenger i Python har mange metoder, inndelt på typer av operasjoner
  - Generelt format:
    - `mystring.method(arguments)`
  - Noen metoder tester en streng for spesifikke karaktertrekk
    - Disse metodene er såkalte boolske metoder som returnerer enten `True` hvis betingelsen er sann ellers `False`.

Noen strengteste-metoder:

Method	Description
`isalnum()`	Returns true if the string contains only alphabetic letters or digits and is at least one character in length. Returns false otherwise.
`isalpha()`	Returns true if the string contains only alphabetic letters, and is at least one character in length. Returns false otherwise.
`isdigit()`	Returns true if the string contains only numeric digits and is at least one character in length. Returns false otherwise.
`islower()`	Returns true if all of the alphabetic letters in the string are lowercase, and the string contains at least one alphabetic letter. Returns false otherwise.
`isspace()`	Returns true if the string contains only whitespace characters, and is at least one character in length. Returns false otherwise. (Whitespace characters are spaces, newlines (`\n`), and tabs (`\t`).
`isupper()`	Returns true if all of the alphabetic letters in the string are uppercase, and the string contains at least one alphabetic letter. Returns false otherwise.

Strengmetoder (forts.)

- Noen metoder returnerer en kopi av en streng som det er gjort forandringer på
  - De ”simulerer” at strenger er muterbare
- Sammenlikning av strenger foregår case-sensitivt
  - Store bokstaver skilles fra små bokstaver
  - lower og upper kan brukes til å gjøre sammenlikning av strenger uten å ta hensyn til *case*

Strengmodifiserings-metoder:

Method	Description
`lower()`	Returns a copy of the string with all alphabetic letters converted to lowercase. Any character that is already lowercase, or is not an alphabetic letter, is unchanged.
`lstrip()`	Returns a copy of the string with all leading whitespace characters removed. Leading whitespace characters are spaces, newlines (`\n`), and tabs (`\t`) that appear at the beginning of the string.
`lstrip(`*`char`*`)`	The *`char`* argument is a string containing a character. Returns a copy of the string with all instances of *`char`* that appear at the beginning of the string removed.
`rstrip()`	Returns a copy of the string with all trailing whitespace characters removed. Trailing whitespace characters are spaces, newlines (`\n`), and tabs (`\t`) that appear at the end of the string.
`rstrip(`*`char`*`)`	The *`char`* argument is a string containing a character. The method returns a copy of the string with all instances of *`char`* that appear at the end of the string removed.
`strip()`	Returns a copy of the string with all leading and trailing whitespace characters removed.
`strip(`*`char`*`)`	Returns a copy of the string with all instances of *`char`* that appear at the beginning and the end of the string removed.
`upper()`	Returns a copy of the string with all alphabetic letters converted to uppercase. Any character that is already uppercase, or is not an alphabetic letter, is unchanged.

Sammenlikningsmetoder

- Programmer trenger ofte å søke etter substrenger
- Flere metoder finnes for å gjøre dette:
  - `endswith(substring)`: sjekker om en streng slutter med substring
    - Returnerer `True` eller `False`
  - `startswith(substring)`: sjekker om en streng begynner med substring
    - Returnerer `True` eller `False`

Søk- og erstatt-metoder

- Flere metoder finnes (forts.)
- `find(substring)`: leter etter substring i strengen
  - Returnerer den laveste indeksen av substrengen, eller -1 om substrengen ikke finnes i strengen
- `replace(substring,new_string)`:
  - Returnerer en kopi av strengen hvor alle forekomster av substring er erstattet med new_string

Oversikt over søk- og erstattmetoder

Method	Description
`endswith(substring)`	The *substring* argument is a string. The method returns true if the string ends with *substring*.
`find(substring)`	The *substring* argument is a string. The method returns the lowest index in the string where *substring* is found. If *substring* is not found, the method returns −1.
`replace(old, new)`	The *old* and *new* arguments are both strings. The method returns a copy of the string with all instances of *old* replaced by *new*.
`startswith(substring)`	The *substring* argument is a string. The method returns true if the string starts with *substring*.

Å splitte opp en streng

- Metoden `split`: returnerer en liste som inneholder ordene i strengen
  - Bruker space (mellomrom) som skilletegn som standard
  - Kan bruke et annet skilletegn gjennom å sende det som argument til metoden split
  - Eks:

```
tekst = 'Dette er en test'
print(tekst.split())
```

Gir:

```
['Dette', 'er', 'en', 'test']
```

Oppgave: overskrift



- Lag funksjonen `overskrift`, som tar inn en parameteren `tekst` og returnerer samme tekststreng der alle ordene starter med stor forbokstav.
  - Benytt: split(), upper(), for-løkke

Bygge strenger med join()

- Metoden `join` tar som input en liste (iterable) med strenger, og limer dem sammen til en streng:

```
navneliste = ["Even", "Ada", "Kaisa"]
",".join(navneliste)
```

- Strengene i listen konkateneres, limt sammen med komma mellom hver. Mye enklere enn f.eks å bruke en løkke til å sette sammen ordene:

```
def lime(liste, lim):
    if len(liste) == 0:
        return ""
    streng = liste[0]
    for i in range(1,len(liste)):
        streng += lim + liste[i]
    return streng
```

Skrive og lese data fra fil

- numpy.loadtxt()
  - Leser data fra fil og konverterer til tall hvis mulig.
  - Resultatet (returverdien til funksjonen) er en numpy-array med dataene.
- Numpy.savetxt()
  - Skriver data fra en numpy-array til fil.
  - Dette representerer tall som tekst

Data representert som tekst

- Vi bruker tekst til kommunikasjon, og representerer mange typer data som tekst, f.eks. tall
- F.eks. så lagres en float som 64 bits internt, men når vi printer tallet, så konverteres det til en sekvens av tegn som representerer tallet, som er tilpasset oss (mennesker).
- Hvis vi skal lagre tall i en fil for å kunne leses senere, eller av et annet program, så kan vi bruke den interne representasjonen («binær fil»), eller bruke en tekst-representasjon (da får vi en tekst-fil, selv om dataene er tall)

numpy.savetxt()

```
numpy.savetxt(fname, X, fmt='%.18e',
   delimiter=' ', newline='\n', header='',
   footer='', comments='# ', encoding=None)
```

numpy.savetxt() - Eksempler

Se egen notebook

numpy.loadtxt()

```
numpy.loadtxt(fname, dtype=<class 'float'>,
comments='#', delimiter=None,
converters=None, skiprows=0, usecols=None,
unpack=False, ndmin=0, encoding='bytes',
max_rows=None, *, quotechar=None, like=None)
```

numpy.loadtxt() - eksempler

- Se notebook

Oppsummering av strenger

- Operasjoner på strenger, som
  - Metoder for å iterere gjennom strenger
  - Operatorer for repetisjon og kontatenering
  - Strenger som ikke-muterbare objekter
  - Å *slice* og teste strenger
  - Metoder for undersøke egenskaper til strenger
  - Metoder for å splitte opp og lage modifiserte strenger (husk: Strenger kan ikke endres)